

HKFilmMap: Exploring Hong Kong film locations through an interactive map

Yikai Wu, Yu Cai, Junkai Meng, Anran Chen.

Group Part

1 Introduction

HKFilmMap is an Android location-based services (LBS) application designed for exploring Hong Kong film locations through an interactive map and a route-planning workflow. The app connects film information, filming places, scene descriptions, map-based points of interest, half-day route generation, check-ins, achievements, suggestions about food or coffee to nearby location.

The project aims to transform static film-location data into an experience that users can actually experience within the city. Through logging in, a user can browse the places of Hong Kong films on the map. Users can log in, browse Hong Kong film places on a map, open details about movies and scenes, add preferred places to a personal route, generate a half-day film route, refine the route through search and list editing, and then use in-app route preview, check-ins, and recommendations during the trip.

The existing packaged seed contains 68 movies, 97 places, and 142 scene records. Among them, 85 scenes are ready to be displayed on map as they contains usable coordinates.



Fig.1. The overall layout of HKFilmMap

2 Target Users and Use Cases

The application has three primary target users: movie buffs that seek specific locations related to their favourite films; out-of-towners who are looking for curated half-day itineraries with cultural themes; and locals who want to re-explore familiar places through the lens of history and cinema.

In order to serve these diverse needs, the main user experience is conceived as a progressive spatial journey. The primary user journey under construction is **Fig. 2.**

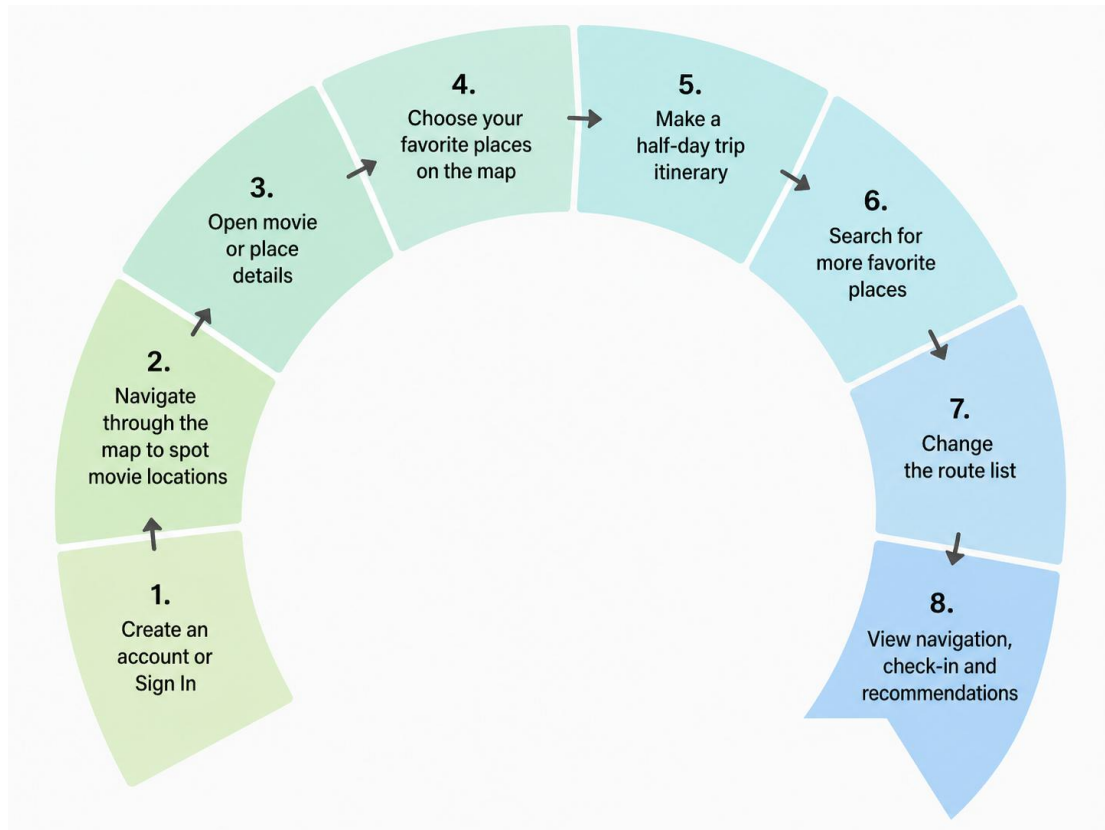


Fig.2. The designed main user journey

And the app serves as an engaging spatial tool that allows a user to create their own individual cinematic routes and physically move through the city.

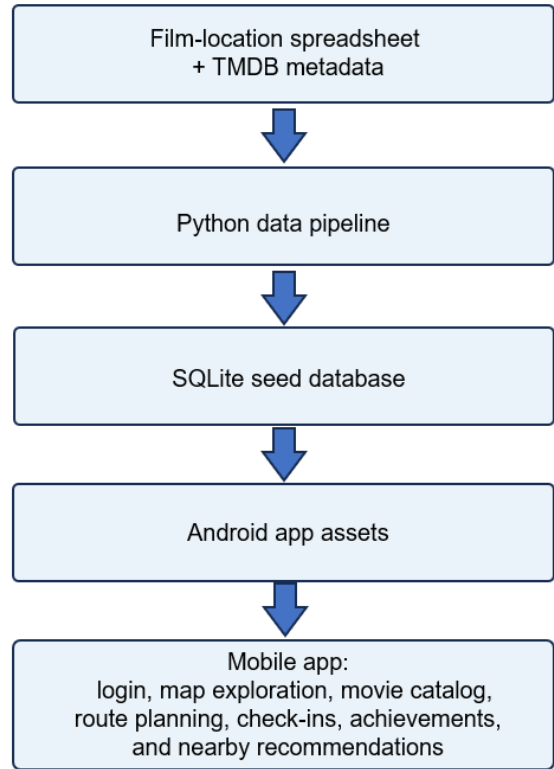
3 System Design

3.1 System Overview

HKFilmMap has two key components.

1. The user interface, map, authorization, route planning, check-in, and suggestions are all included in the Android application.
2. The backend processing of data to prepare the film-location dataset and produce the app's SQLite seed database.

42 The overall system flow is:



43
44 **Fig. 3. Overall system workflow**

45
46 The Android app development uses technologies as detailed in **Table 1:**

47
48 **Table 1. Technologies used in the HKFilmMap development**

Technology	Role
Firebase Authentication	User sign in, register and email link sign in.
Google Maps	Main map interface and showing location.
Google Directions API	Route preview with moving polyline
SQLite	We provide local storage for various locations and movies.
Python scripts	Cleaning up data, adding information and generating the database.

49
50 **3.2 Data Design**

51 This project employs an automated pipeline which transforms the raw location data into
52 a local SQLite database that is directly embedded in the Android application for ease
53 of use. To begin with reading the local override files and the initial spreadsheet. Next

step, removal of duplicate instances of each movie and verification of spatial coordinates of every location. The dataset includes movie metadata and poster images sourced from The Movie Database (TMDB) API (<https://developer.themoviedb.org/docs/getting-started>). A key feature of the final data base is easy identification of Movies, Places, and Scenes. A Scene can basically be described as a link between a movie and a real-world location on the map. Breaking the two activities apart directly supports two app functions: route planning being more place-based while user check-ins are more scene-based. The data can be exported to the app after verification of its consistency and coordinates. By doing this, both the map and film data can be updated individually without triggering the live back-end server.

3.3 Layout and Interaction Design

The application employs a structured, task-oriented layout design to provide an intuitive and user-friendly experience as shown in the **Table 2** and the figure. User journey starts with simple Login page (**Fig. 4.A**). The main Explore function (**Fig. 4. B**) has a “full map + bottom sheet” layout which makes optimal use of space to show the movies which are filmed in Hong Kong. Furthermore, users can also filter the piers at the bottom menu.

The Route Planner is used for journey mapping (**Fig. 4. C and E**) and comprises several sections: a configuration card to generate routes, a dynamic map view, and a list of selected scenes. At the bottom of this screen, there is a fixed “Start Navigation” button for easy operation. On the other hand, users wanting to browse by film rather than using chips can use the Movie Catalog (see **Fig. 4. D**), a vertically scrollable list of genre chips. When users want to find out more, topic-specific detail pages combine general header information with a topic scene list and a localized map preview. Lastly, a list of achievements on the Achievements screen (**Fig. 4. F**) gives users a sense of progress through exploration and scene check-ins.

Table 2. Layout and Interaction Design

Screen	Layout choice
Login or Register	Form-based page
Explore	Full map plus bottom sheet
Route Planner	A configuration card, map preview, tab list, and fixed bottom actions
Movie Catalog	Scrollable movie list with genre filters.
Detail pages	A map preview and scene list including header details.
Achievements	Progress-style list

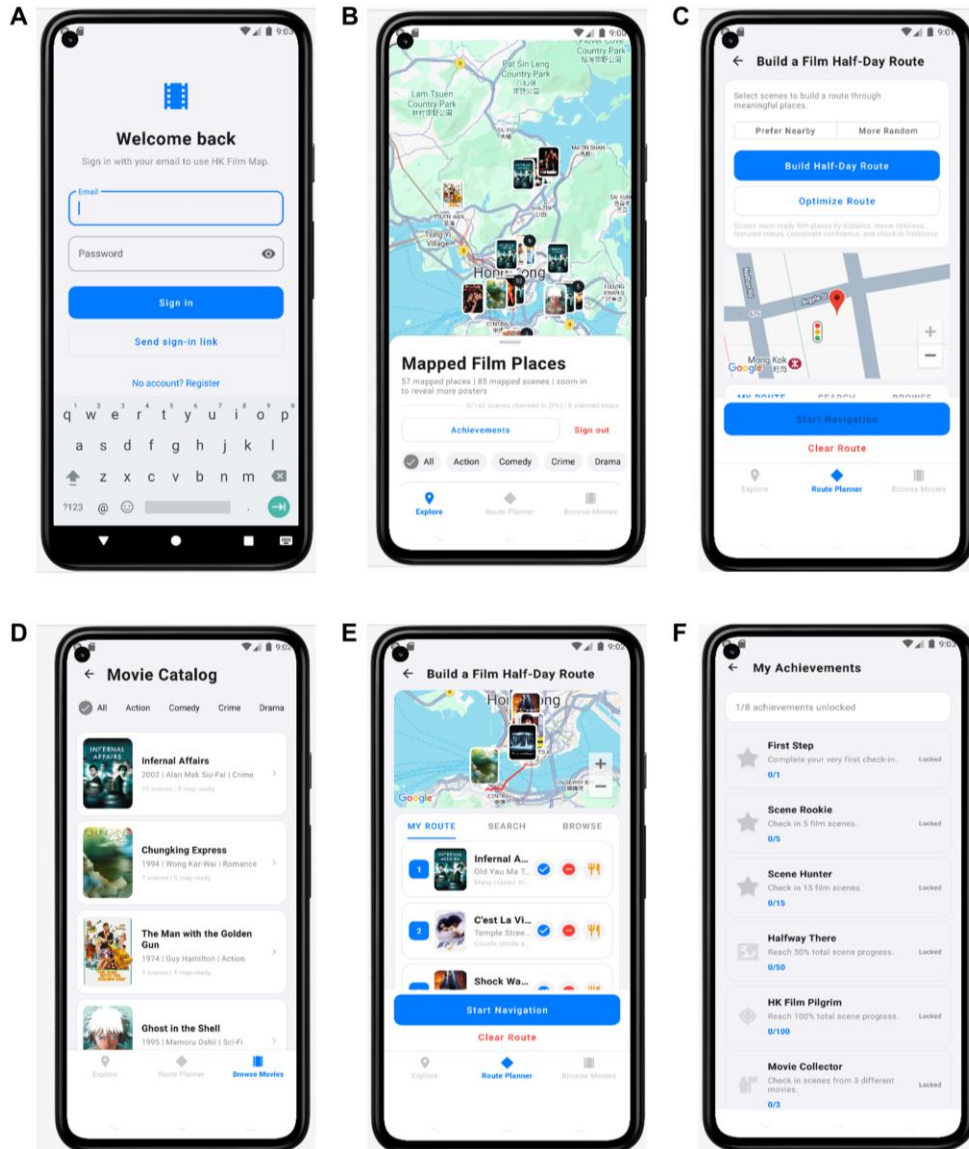


Fig. 4. Layout Design

3.4 Main Algorithm Design

The section details the main spatial and recommendation algorithms of the application, dynamic mapping, path generation, spatial optimization and contextual recommendation.

3.4.1 Map Marker Display Algorithm

To avoid visual clutter within the urban context, zooming in will see a change in the map marker. At lower zooms, nearby places are grouped into cluster markers based on screen grid cells, not distance, to avoid seeing the same marker in multiple places on the device. As we zoom into the map, the clusters dissolve into individual poster stack markers of specific places. As a user zooms in to the highest level, the marker will fan out showing the individual movie posters if one place has scenes from many movies so

the user is able to know the specific film location.

Table 3. Map Marker Display

Zoom level	Display
Low zoom	Cluster markers for places nearby.
Medium zoom	A poster-stack marker should be shown for each place.
High zoom	Spread out movie poster markers to locations with multiple movies.

Pseudocode:

```
1. Input:
2.   places: filtered film places
3.   zoom: current map zoom level
4. Output:
5.   markers drawn on the map
6.
7. 1. If zoom < clusterThreshold:
8.   clusters <- BuildScreenGridClusters(places, zoom)
9.   For each cluster:
10.    If cluster has only one place:
11.     DrawPosterStackMarker(cluster.place)
12.    Else:
13.     posters <- SamplePosters(cluster, limit = 3)
14.     DrawClusterMarker(cluster.center, posters,
15.                        cluster.movieCount, cluster.sceneCount)
16. 2. Else if zoom < expandedThreshold:
17.   For each place in places:
18.    DrawPosterStackMarker(place)
19. 3. Else:
20.   For each place in places:
21.    scenes <- GetUniqueMovieScenes(place)
22.    If scenes is empty:
23.     DrawPosterStackMarker(place)
24.    Else:
25.     positions <- BuildFanOutPositions(place.center, scenes.count)
26.     For each scene and matching position:
27.      DrawPosterMarker(position, scene.poster)
```

3.4.2 Half-Day Route Generation Algorithm

The route generation algorithm engineers a personalized half-day itinerary. The candidate location is scored based on multiple criteria, which will favour places that are geographically close, contextually relevant to the user's selected genre or theme, and feature-rich. The score also prioritises locations that have not been visited and which have verified spatial coordinates with controlled randomness. The route generated assumes the candidates are selected by enforcing a stringent time budget from which the actual route is generated. It assumes about 25 minutes of visit time per stop plus estimated travel time.

Pseudocode:

```
1. Input:
2.   origin: start location
3.   mode: Prefer Nearby or More Random
4.   genre: selected genre
5.   durationHours: planned route duration
6.   minStops, maxStops: stop count limits
7. Output:
8.   ordered route stops
9.
10. 1. previousRoute <- Save current route scene IDs.
11. 2. Clear current draft route.
12. 3. targetMinutes <- Clamp(durationHours * 60, 180, 300).
13. 4. candidates <- SearchRouteCandidates(query = "", genre, limit = 240).
14. 5. scoredCandidates <- empty list.
15. 6. For each candidate in candidates:
16.     If candidate has no coordinates:
17.         Continue.
18.     If candidate is already in the route:
19.         Continue.
20.     score <- ScoreCandidate(candidate, origin, mode, genre).
21.     Add (candidate, score) to scoredCandidates.
22. 7. Sort scoredCandidates by score in descending order.
23. 8. selected <- ChooseCandidatesByTimeBudget(scoredCandidates,
24.                                             origin,
25.                                             targetMinutes,
26.                                             minStops,
27.                                             maxStops).
28. 9. selected <- RemoveWeakCandidatesIfOverBudget(selected, targetMinutes).
29. 10. If selected is empty:
30.     Restore(previousRoute).
31.     Return empty list.
```

```

32. 11. ordered <- OptimizeRouteOrder(origin, selected).
33. 12. Insert ordered stops into route_plan_stops with visitOrder.
34. 13. Return ordered.

```

3.4.3 Route Order Optimization

After the selection of route stops, the order of visits is optimized so as to minimize the total distance travelled. In other words, the simplified version of Travelling Salesperson Problem is solved by the system with no return to origin requirement. The Haversine method calculates spatial distances thanks to the sphere shape of the Earth.

A hybrid optimization strategy is adopted for geometric precision improvement and mobile processing performance.

We use exact dynamic programming for routes with 10 or fewer stops to guarantee the absolute shortest path as defined by the state transition.

Pseudocode:

```

1. Input:
2.   origin: start coordinate
3.   stops: route stops, n <= 10
4. Output:
5.   stops in optimized order
6.
7. 1. Compute dist[i][j] for all pairs of stops.
8. 2. Compute originDist[i] from origin to each stop.
9. 3. Initialize dp[mask][end] to Infinity.
10. 4. For each stop i:
11.     dp[1 << i][i] <- originDist[i].
12. 5. For each mask:
13.     For each end included in mask:
14.         prevMask <- mask without end.
15.         For each prev included in prevMask:
16.             candidate <- dp[prevMask][prev] + dist[prev][end].
17.             If candidate < dp[mask][end]:
18.                 dp[mask][end] <- candidate.
19.                 parent[mask][end] <- prev.
20. 6. Select the end with minimum dp[fullMask][end].
21. 7. Backtrack through parent to rebuild the route order.
22. 8. Return the ordered stops.

```

For longer routes, it's harder to calculate the exact path, so the program will switch to a faster heuristic calculation that will make use of a nearest neighbor algorithm and then

210 do some 2-opt swaps.

211 **Pseudocode:**

```
212 1. Input:  
213 2. origin: start coordinate  
214 3. stops: route stops, n > 10  
215 4. Output:  
216 5. approximately optimized route order  
217 6.  
218 7. 1. route <- empty list.  
219 8. 2. remaining <- all stops.  
220 9. 3. current <- origin.  
221 10. 4. While remaining is not empty:  
222 11.     next <- stop in remaining with minimum distance from current.  
223 12.     Add next to route.  
224 13.     Remove next from remaining.  
225 14.     current <- next.  
226 15. 5. improved <- true.  
227 16. 6. While improved:  
228 17.     improved <- false.  
229 18.     For each pair of indices i, j:  
230 19.         before <- distance of affected edges before reversing route[i..j].  
231 20.         Reverse route[i..j].  
232 21.         after <- distance of affected edges after reversal.  
233 22.         If after < before:  
234 23.             improved <- true.  
235 24.         Else:  
236 25.             Reverse route[i..j] again to undo the change.  
237 26. 7. Return route.
```

238

239 **3.4.4 Search and Route Deduplication**

240 The search function aggregates by physical places rather than by discrete movie shots
241 in the interest of spatial routing. This means that when one place comes into more than
242 one movie it removes other listings and provides the location only once.

243 **Pseudocode:**

```
244 1. Input:  
245 2. query: user keyword  
246 3. genre: selected genre  
247 4. limit: maximum number of results  
248 5. Output:  
249 6. candidate places
```

```

250 7.
251 8. 1. normalizedQuery <- Trim(query).
252 9. 2. If normalizedQuery is empty:
253 10.     likeQuery <- "".
254 11.     Else:
255 12.     likeQuery <- "%" + normalizedQuery + "%".
256 13. 3. Query movies, places, and scenes.
257 14. 4. Match fields including:
258 15.     place name,
259 16.     district,
260 17.     address,
261 18.     movie English title,
262 19.     movie Chinese title,
263 20.     genre,
264 21.     scene title,
265 22.     scene description.
266 23. 5. Group results by placeId.
267 24. 6. Compute movieCount, sceneCount, hasFeaturedMovie,
268 25.     representative poster, top movie title, and isInRoute.
269 26. 7. Sort by:
270 27.     isInRoute DESC,
271 28.     hasFeaturedMovie DESC,
272 29.     movieCount DESC,
273 30.     placeName ASC.
274 31. 8. Return the first limit candidates.

```

275

276 When a user selects this place in their itinerary, then algorithm will choose a
277 representative scene, with valid coordinates, to anchor the map stop. Because of this
278 de-duplication based on place, the system does not create spatial routing for the same
279 physical destination repeatedly.

280 Pseudocode:

```

281 1. Input:
282 2.     placeId
283 3. Output:
284 4.     true if added, false otherwise
285 5.
286 6. 1. If the route already contains this place:
287 7.     Return false.
288 8. 2. sceneId <- FindRepresentativeMappableScene(placeId).
289 9. 3. If sceneId does not exist:
290 10.     Return false.

```

```

11. 4. visitOrder <- Current maximum visitOrder + 1.
12. 5. Insert (planId, sceneId, visitOrder) into route_plan_stops.
13. 6. Return true.

```

3.4.5 Nearby Recommendation Ranking

The recommendation engine suggests nearby amenities that could serve as the film tour's logical breaks. Instead of just giving the closest geographic options, this approach uses an expanding spatial search radius (from 1.2 km up to 4.5 km), and ranks its candidates using a composite score. This score takes into account how close it is spatially, whether the correct district has been matched, time of day, price point, and even the “mood” of the movie associated with that time. At the last ranking, a category penalty is applied to guarantee diversity. Without this penalty, the system would just suggest one type of amenity, denying users exposure to any alternatives.

Pseudocode:

```

1. Input:
2.   origin: current film place
3.   districtHint: district of the current place
4.   genreHint: movie genre
5.   limit: number of recommendations
6. Output:
7.   ranked recommendation list
8.
9. 1. candidates <- empty list.
10. 2. For radius in [1.2km, 2.0km, 3.2km, 4.5km]:
11.   For each item in nearby_food.json:
12.     distance <- Haversine(origin, item.location).
13.     If distance <= radius and item is not duplicated:
14.       Add item to candidates.
15.     If candidates.count >= limit:
16.       Break.
17. 3. If candidates are not enough:
18.     Add same-district fallback items.
19. 4. If candidates are still not enough:
20.     Add nearest fallback items.
21. 5. For each candidate:
22.   score <- DistanceScore(distance)
23.     + DistrictMatchBonus(candidate, districtHint)
24.     + TimeOfDayBonus(candidate.category, currentHour)
25.     + GenreMoodBonus(candidate.category, genreHint)
26.     + PriceBonus(candidate.priceRange)
27.     + EditorialBoost(candidate.description).

```

28. 6. Sort candidates by score in descending order.
29. 7. Diversify the list by applying a category penalty
30. when too many selected items have the same category.
31. 8. Assign rank values.
32. 9. Return the ranked list.

4 Main Functions

4.1 Login and Registration

The application begins with secure authentication process. After being authenticated the users are directed towards the main spatial interface Explore Map. Utilizing a full-screen cartographic layout integrated with an interactive bottom sheet, this interface is optimized for non-linear urban exploration. Users can visually navigate the spatial distribution of films' locations across Hong Kong, apply thematic filters according to their preferred movie genres, and access detailed metadata for specific POIs. This design connects physical geography and culture. Users can view film snippets linked to the site, register a spatial check-in or place a site on their personal itinerary.

Sign-in screenshots omitted from the public edition
because they contain account information.
The original coursework submission is retained privately.

Fig. 5. User Authentication Flow and Secure Sign-in Interface

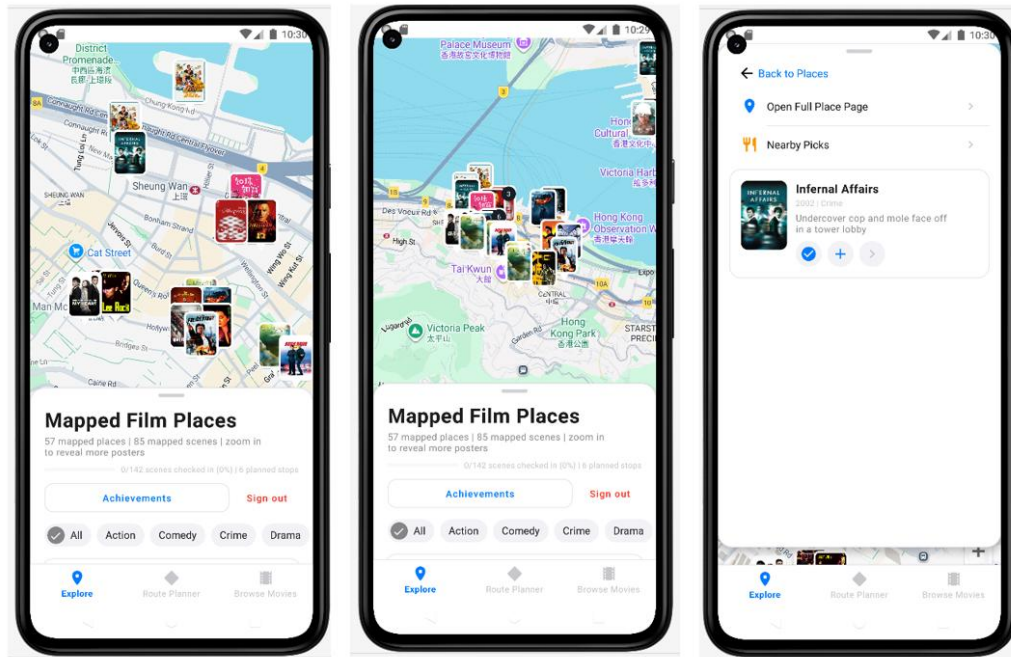


Fig. 6. POI Details

4.2 Explore Map

Both spatial-first and thematic-first discovery are enabled by the system to cater to user behavior. User can either make a move using the geographic place to the film or back through the featured film to the world where it was actually filmed. The Route Planner provides a flexible spatial workspace for active journey mapping. Users have two options to create an itinerary; either creation through POIs selected in the map or catalog or allowing the system to produce a pre-packed half-day itinerary. The planner has a configuration panel, live map preview, and interactive itinerary list. Users can drag-and-drop to adjust the order of stops, or they can automatically optimize the route for the system to follow before moving.

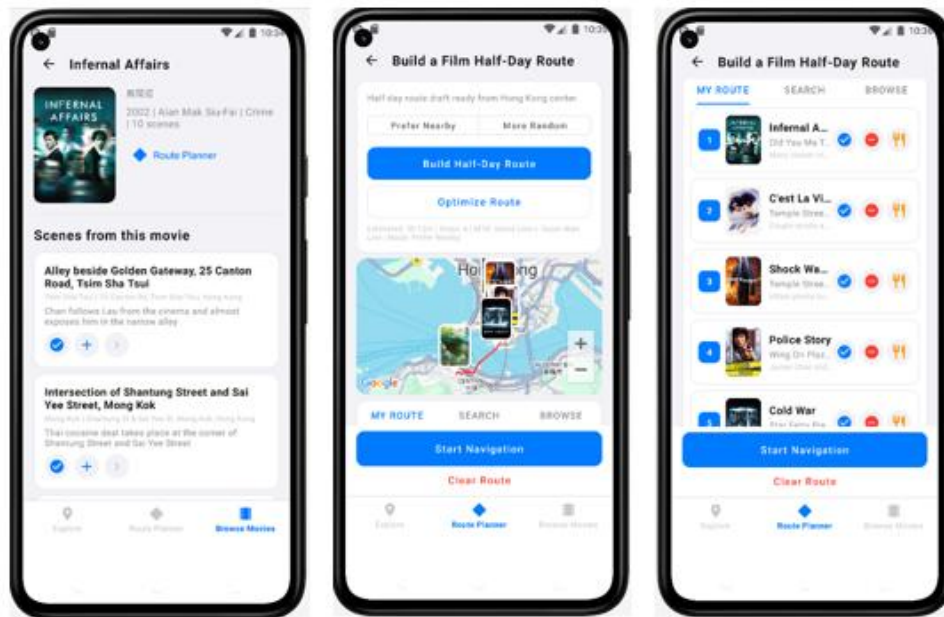


Fig. 7. Interfaces for Thematic Discovery and Dynamic Route Generation

4.3 Check-ins, Achievements, and Recommendations

To encourage physical interaction with the city, the app includes location-based gaming. Users can keep track of their geographic exploration and progress in achievement unlocks (e.g. after reaching a certain number of check-ins or completing a multi-destination route) through physical ‘check-ins’ at spots in the film. To further improve urban tourism development, a context-aware recommendation engine will be utilized. After checking in, the application suggests nearby dining or cafes in the area. These recommendations are based on location and the time of the day, while also taking into account the theme of the particular film.

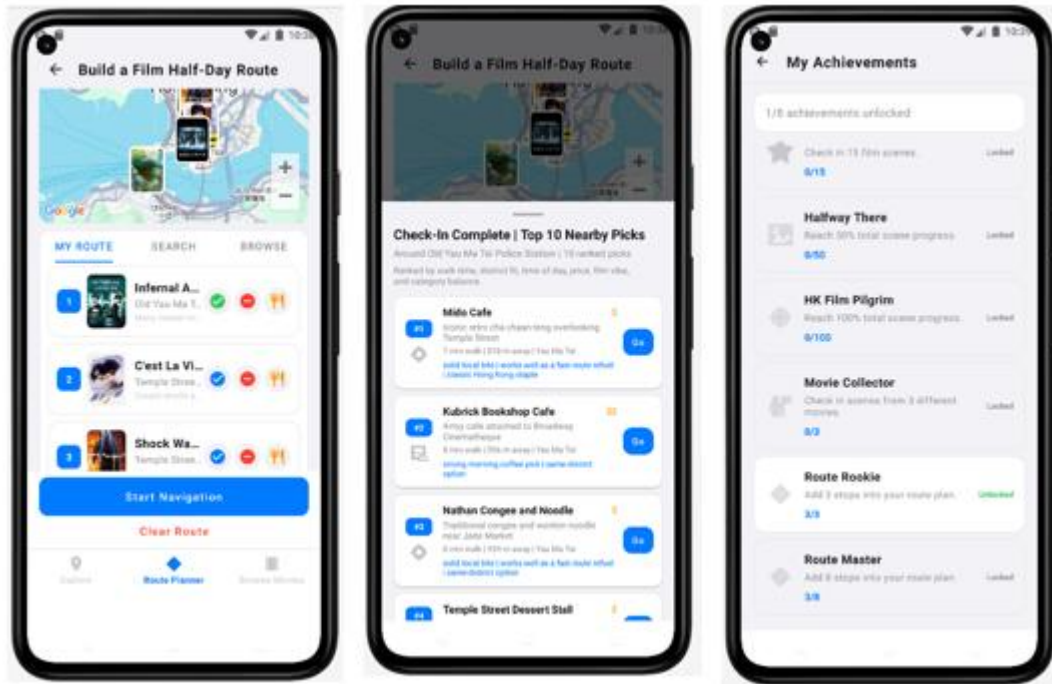


Fig. 8. User Interfaces for Spatial Check-ins, Dynamic Amenity Recommendations, and Milestone Tracking

5 Significance of the App

1. HKFilmMap highlights the connection between the digital and the physical, the cinematic and the geographical. It highlights that films are not just on-screen fictions; they are moving picture archives of the real city. The city's unique built environment and its dense urban morphology are identified as significant elements in the cinematic character and ambiance of Hong Kong cinema.
2. Organized, thematic tourism will be enabled through the app and more likely to happen. Users will use mapped itineraries that take them through sites continuously related by particular movies, genres, or neighbourhoods rather than disjointed and generic sites. Travelling isn't simply arriving at your destination but connecting and bonding with your final destination.

6 Potential Profit and Sustainability Model

The long-term sustainability depends on the use of spatial data that can encourage cultural tourism, academic research and institutional partnerships, and promote heritage over commercial traffic.

1. Partnering with tourism boards, film festivals, and heritage organizations to designate official cinematic routes.
2. Creation of itineraries of a specific spatial nature (director-oriented, district-

- 399 specific etc.) capable of being sponsored by an organization.
- 400 3. Local Enterprise Integration ensures grounding of the context-aware promotions
401 on these paths in connection with services available to the currents. For example,
402 the solution may provide users with coupons for the nearby urban amenities (e.g.,
403 local cafes or restaurants). To ensure that the algorithm remains neutral and does
404 not lose consumer trust, the algorithmic recommendation rankings must prioritize
405 space and theme over payment.
- 406 4. Top quality contextual content is offline spatial guides, director's commentary,
407 historical context content and similar additions that add high value.
- 408 5. The organized and geocoded database of film locations can be packaged and
409 licensed to aid scholar researchers, urban planners, and tourism organizations
410 outside of the film industry.

411

412 **7 Limitations and Future Work**

413 **7.1 Limitations**

414 While this application does an excellent job of making your movement through urban
415 spaces feel like film, its architecture does have some limitations when it comes to
416 processing dynamic data. The route production algorithm is primarily based on static
417 transit proxies to estimate travel time and not on real-time Directions APIs.
418 Consequently, the Estimated Time of Arrival (ETA) between spatial nodes serves as a
419 heuristic approximation rather than a definitive transport-planning metric.

420 In addition, the context-aware recommendation engine is currently limited to bounded
421 local JSON datasets. Doing this offline restricts the degree to which nearby amenity
422 recommendations can scale. It simply isn't connected to live, dynamic urban data
423 streams, such as operating hours (open/closed) or live ratings by actual users.

424

425 **7.2 Future Work**

426 To combat these urban limitations and develop a fruitful tourism experience, future
427 plans will focus on three crucial elements.

- 428 1. This integration will enable the service to provide passengers with up-to-date
429 arrival times for buses. Moving away from static proxies and turning to the real-
430 time data from networks will improve spatial routing, dynamic ETAs and
431 reliability in general.
- 432 2. To enhance the value of cinematic tourism, the platform will promote pre-curated
433 "themed routes". These itineraries say, the Wong Kar-wai route or the harbour route
434 will be specially designed to offer users highly focused spatially and culturally
435 immersive narratives.

- 436 3. To support saved route collections and community sharing functionality that
437 promote long-term use, future implementations will this collaborative spatial
438 networking on infrastructure level (cloud). Users are allowed to publish along with
439 exchange their custom itineraries which will convert the application from a one-
440 user routing app into an app that is crowd-sourced for a city.

441

442

Personal Part

Yikai Wu

Student ID:

Role: Coding (Algorithm & Database) + Report Lead

For the HKFilmMap project, I was mainly involved with coding, design of the spatial algorithms, and the report materials.

I was involved in writing and debugging code as a core developer. I was in charge of a multi-level spatial visualization design for movies' location data, showing the location data in a geographical format. Additionally, I designed the main recommendation algorithms and implemented them for features such as personalized destination recommendations and automated half-day routes. I created the database schema to facilitate these computational functions. I managed the academic deliverables as the lead which included the literature review and complete writing and editing of the final project report apart from the technical development.

The initial database construction posed the most important technical challenge I faced. Sourcing and verifying in-depth film data, locations and promotional images manually were tedious and non-scalable activities. To solve this data bottleneck I researched and integrated The TMDB API into our backend pipeline. As the data enrichment process is automated, it populated our local seed database with strong, high-quality data.

Yet another tech issue was debugging the UI layout of the interactive Mapped Film Places pull-up window. At first, providing the drag experience as well as layout constraints on the underlying map view caused visual glitches and hierarchy issues. Through continuous consultation with developer communities CSDN and AI assistance, I could resolve the layout issues that came up and make the interface stable.

In general, this project sharpened my overall skills, covering the front-end of spatial visualisation, the back-end of algorithmic logic and the academic writing. According to the results, strong data preparation and a clear spatial design are the two key components that make a LBS application valuable for city exploration.

474 **Yu Cai**

475 Student ID:

476 Role: Coding and presentation materials

477

478 For the group project associated with LSGI541, I selected the Coding + Presentation
479 Role which is noted in our team workload document. My major focus was on the
480 Explore (map) module, the movie catalog and navigation into place and movie detail
481 screens – the link between the data in Room/SQLite and what the user sees on the
482 map and in lists.

483 At the implementation stage, I put in place genre filtering, refreshing markers when the
484 filter changes, and wired the bottom sheet so that pressing a marker or row shows the
485 place title, scene list and state. The most challenging aspect of the game was the zoom-
486 dependent rendering functionality in the MainActivity. In the application, at low zoom
487 we cluster POIs with a screen-space based grid to prevent overlapping. Whereas, at
488 medium zoom we use a poster stack marker for each location. Furthermore, at high
489 zoom we fan out separate markers per film around the same point using ring-shaped
490 offsets in pixel-space. Finally, we map back to lat-long with Projection. The
491 construction of that pipeline was much more than just “one marker per row” and
492 required careful handling of onCameraIdle refresh.

493 I encountered difficulties whilst trying to balance the visual richness against scrolling
494 smoothness, differences between the emulator and the real device for location
495 permission and camera callbacks, and trying to keep the RecyclerView adapter params
496 aligned with the DAO when the filter changes. For example, a simple mistake would
497 cause the lists to be empty. In the oral presentation, information about the project
498 background, the target users, and the end-to-end workflow from maps browsing to route
499 planning was contributed by me. Emphasis was put on the difference between a film
500 pilgrimage LBS and a normal navigation.

501 It helped me hone in on Android UI lifecycle, map-first UX, demystifying spatial
502 features to non-developers. I realised that the HKFilmMap first screen must be based
503 on geography and easy to read. Without that users never discover routing, check-in or
504 nearby picks. Both map interaction quality and data quality are essential for Mobile
505 GIS to be effective

506

507 **Anran Chen**

508 student ID

509 Role: PPT Design + Report

510

511 I designed and made the final presentation slides in the HKFilmMap project. I created
512 easy to understand documentation around the teams technical, product logic and overall
513 processes. This task did not only involve gathering material from the members. In
514 addition, it included reorganizing the content to help clearly communicate what the
515 project does and what value it can provide to a layman.

516 Initially, I organized the entire content into one complete story, starting from the user
517 scenario. I wanted to make it known that our project is more than just showing movie
518 info or a simple map. It helps users transition from discovering film locations to
519 planning a half-day route, before extending to related features like navigation, check-
520 in and nearby suggestions. The overall framework consists of problem background, data
521 issues, system design, core functions, major innovations and future works. I also made
522 sure the words and visuals were the same throughout to give it greater organization.

523 With this framework, I organized and reviewed the main technical aspects of the project
524 which were data preprocessing, system architecture, algorithm, and user flow, etc. One
525 major challenge I faced was ensuring technical depth and clarity. The project consists
526 of complicated elements such as data cleaning, heuristic route algorithms and
527 recommendation systems. Showing all of the technical details will make it difficult for
528 the audience to grasp. So in order to tackle this, I simplified and summarised the gist
529 using diagrams, pointers and data comparison. Through this, the way it was done and
530 what was achieved were made clear for better communication.

531 As I worked through this, I was able to enhance my ability to communicate technical
532 projects clearly, and organize complex information better, which is quite vital for the
533 presentation and teamwork across sectors.

534

535 **Junkai Meng**

536 Student ID is .

537 Role: Video Demo and Presentation

538

539 I was responsible for the Video Demo and Presentation for LSGI541 Group Project My
540 main task was to create a screen-recording video that illustrates the overall app
541 workflow, starting from installation and first launch to all the major features and the
542 PowerPoint slides used during the presentation.

543 For the demo video, a key challenge was designing a recording sequence that gave an
544 organic feel rather than a random click-through. I followed the basic user flows of the
545 product. Furthermore, I explored the map, added places to a route, generated a half-day
546 itinerary, searched and edited stops, navigated and checked in, and finally viewed the
547 nearby Top 10 food and coffee recommendations. Every transition had to be paced so
548 that audiences would really be able to read everything on screen without the video
549 dragging. I faced some practical challenges too. I had to configure the proxy for the
550 Android emulator's network. This way, I would be able to load Firebase authentication
551 and Maps properly while working in mainland China. I had to choose the right Profile
552 of the emulator device.

553 I avoided any reference to the implementation, instead telling the story through an end-
554 user scenario. The story progressed from the issue (film locations are all over the place
555 and difficult to plan), through the data pipeline (2 092 raw rows cleaned to 142 usable
556 scenes), to the live demo, and finally to algorithmic highlights like the heuristic half-
557 day route score and diversity aware recommendation ranking. I ensured that each slide
558 had a single point so the audience could follow even without any background on Room
559 databases or map clustering.

560 I learned that the app demonstration is an important part of the development process
561 that requires almost as much preparation as the coding work. If a demo is poorly
562 structured, even good features can be hidden whereas a clear walkthrough can make a
563 simple interaction persuasive. While I improved the quality of my presentation
564 storytelling—focusing on the user impact of the technical decision rather than the
565 implementations.